



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Mesterséges Intelligencia és Rendszertervezés Tanszék

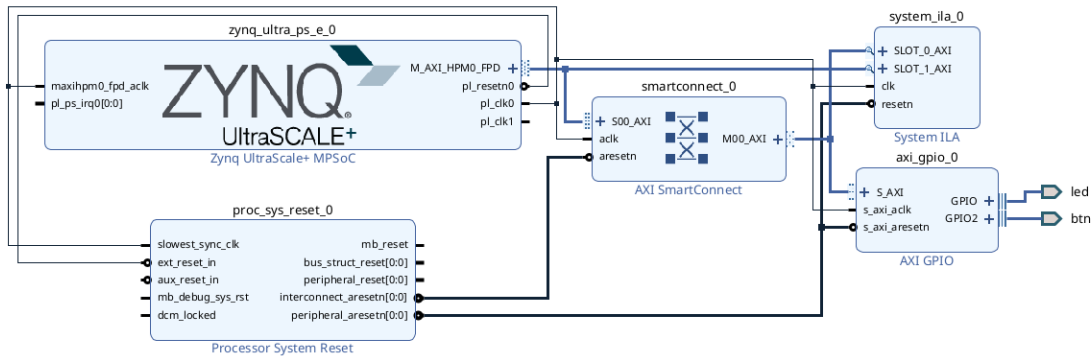
FPGA TERVEZŐI LABORATÓRIUM 2026.

Jedla Martin, DEC4F6

https://github.com/jedlamartin/fpga_labor

1 Egyszerű processzoros rendszer létrehozása

1.1 A rendszer blokkvázlata:



Címkiosztás:

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0 (/zynq_ultra_ps_e_0/Data)					
/zynq_ultra_ps_e_0					
/zynq_ultra_ps_e_0/Data (40 address bits : 0x00A0000000 [256M], 0x0400000000 [4G], 0x1000000000 [224G])					
/axi_gpio_0/S_AXI	S_AXI	Reg	0xA000_0000	64	0xA000_FFFF

1.2 C kód

A nyomógombok állapotát a LED-eken megjelenítő C kód (olvashatóan törölve, syntax highlight-tal, kommentekkel):

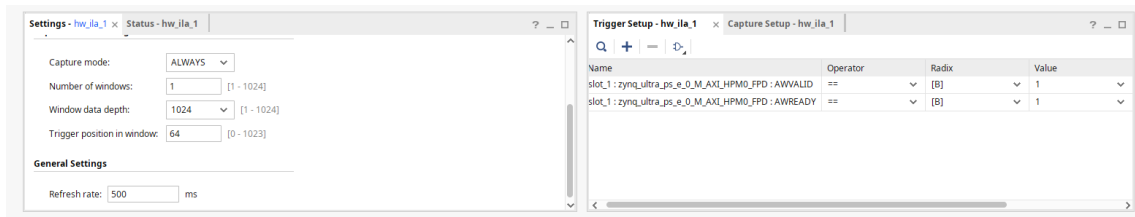
```
#include <xparameters.h>
#include <stdint.h>
#define MEM32(addr) (*(volatile uint32_t *) (addr))

// AXI GPIO leirasbol
#define LED_OFFSET 0x00
#define BTN_OFFSET 0x08

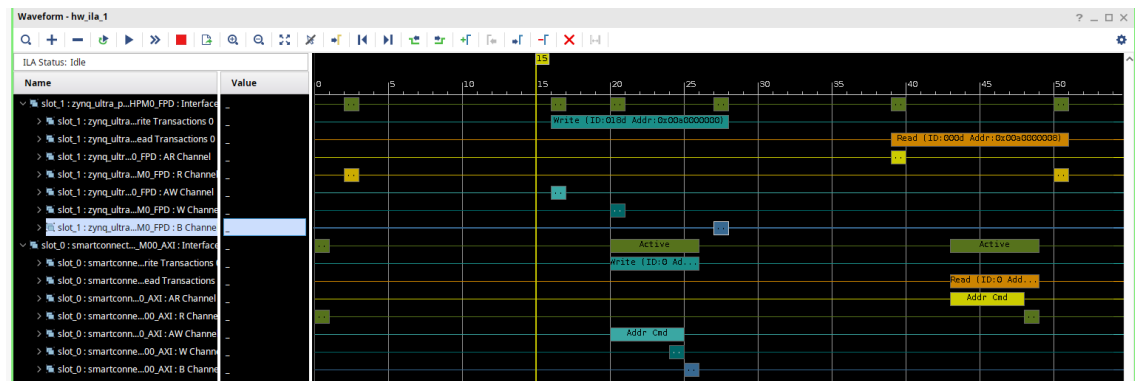
int main() {
    uint32_t data;
    for(;;) {
        data=MEM32(XPAR_AXI_GPIO_0_BASEADDR + BTN_OFFSET);
        MEM32(XPAR_AXI_GPIO_0_BASEADDR + LED_OFFSET) = data;
    }
}
```

1.3 ChipScope hullámformák

1.3.1 AXI4 Lite írási buszciklus:

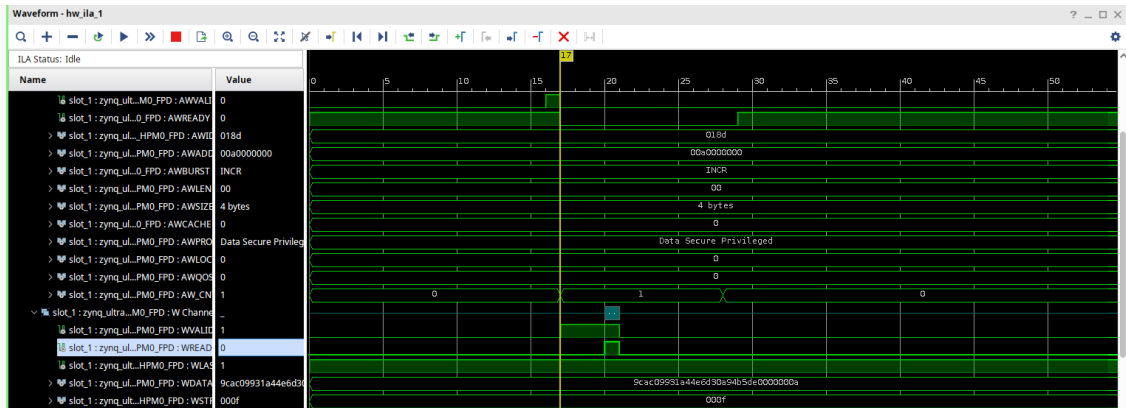


ChipScope hullámforma ÁBRA + részletes magyarázat

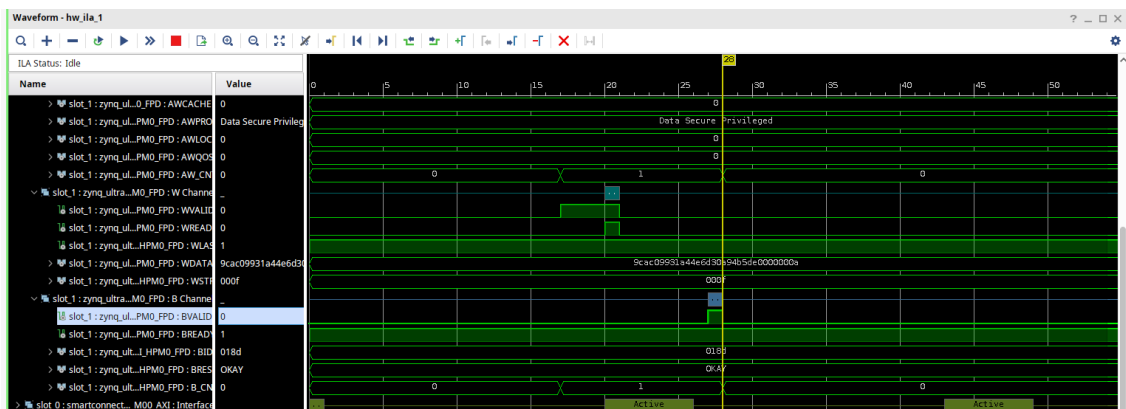


- Az AWVALID jel aktív állapotba kerülésekor az AWREADY jel értéke logikai alacsony szinten marad mindaddig, amíg a periféria részéről meg nem történik a címfogadás nyugtázása.
- A 0. és 2. sorszámú nyomógombok lenyomása következtében a GPIO periféria irányába továbbított adatsínen hexadecimális 0x5 érték figyelhető meg.

Az AXI4-Lite interfész jellemzőinek elemzéséhez integrált logikai analizátorokat (ILA) illesztettem a processzor és az AXI SmartConnect (SLOT1), valamint a SmartConnect és a GPIO periféria közötti csomópontokra. Ez a mérési elrendezés lehetővé teszi a SmartConnect által végrehajtott adatszélesség-konverzió (szélesség-illesztés), továbbá a buszhierarchia által okozott késleltetés (latency) részletes és egzakt vizsgálatát.

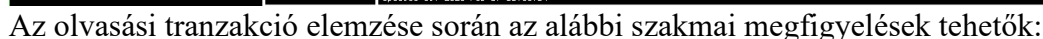


- Az írási tranzakció a címsatornán az AWVALID és AWREADY jelek egyidejű aktív állapotával (handshake) veszi kezdetét.
- Az adatfázis folyamán a WVALID jel jelzi az érvényes írási adat jelenlétét, a tranzakció pedig a WREADY jel magas szintű állapotára fejeződik be.
- A triggerfeltétel teljesülésekor a LED1 és LED3 állapotának megfelelő bemenetek voltak aktívak, így a továbbított adat elvárt értéke binárisan 1010 (hexadecimálisan 0xA, decimálisan 10), amely érték helyesen meg is jelenik a WDATA buszon.
- A processzor és az AXI SmartConnect közötti interfész 128 bites adatszélessége, valamint a GPIO periféria 32 bites interfésze közötti különbség miatt a SmartConnect adatszélesség-konverziót végez.
- Ebből adódóan a 128 bites sín felső, nem használt tartománya nem definiált (úgynevezett „don't care”) adatokat tartalmaz.



Az írási tranzakció lezárásaként a nyugtázási fázis (Write Response) folyamata is pontosan nyomon követhető a vizsgált interfészen:

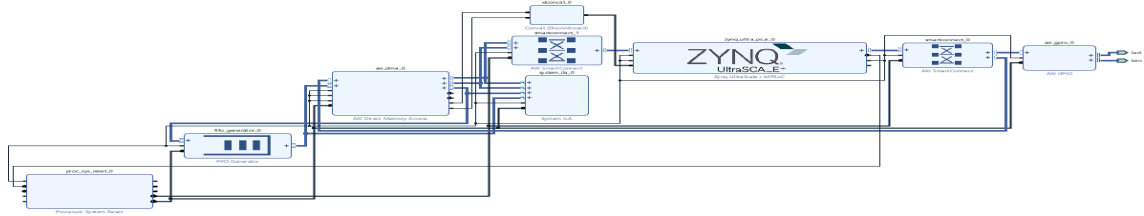
- ### 1.3.2 AXI4 Lite olvasási buszciklus:



- 5

2 DMA vezérlő megvalósítása és tesztelése

2.1 A rendszer blokkvázlata:



Címkiosztás:

Network 0 (/axi_dma_0/Data_MM2S, /axi_dma_0/Data_S2MM)
axi_dma_0
axi_dma_0/Data_MM2S (32 address bits : 4G)
zynq_ultra_ps_e_0/SAXIGP0
S_AXI_HPC0_FPD
HPC0_DDR_LOW
0x0
2G
0x7FFF_FFFF
Excluded (3)
axi_dma_0/Data_S2MM (32 address bits : 4G)
zynq_ultra_ps_e_0/SAXIGP0
S_AXI_HPC0_FPD
HPC0_DDR_LOW
0x0
2G
0x7FFF_FFFF
Excluded (3)
Network 1 (/zynq_ultra_ps_e_0/Data)
zynq_ultra_ps_e_0
zynq_ultra_ps_e_0/Data (40 address bits : 0x0A0000000 [256M], 0x0400000000 [4G], 0x1000000000 [224G])
axi_dma_0/S_AXI_LITE
S_AXI_LITE
Reg
0xA001_0000
64K
0xA001_FFFF
axi_gpio_0/S_AXI
S_AXI
Reg
0xA000_0000
64K
0xA000_FFFF

2.2 A DMA vezérlő regiszterei:

Offszet Cím	Név	A regiszter funkciója (bitek ismertetése, illetve a funkcionalitás szöveges leírása)
0x0	MM2S_DMACR	MM2SDMA Control register
0x4	MM2S_DMASR	MM2SDMA Status register
0x18	MM2S_SA	MM2S Source Address. Lower 32 bits of address.
0x1C	MM2S_SA_MSB	MM2S Source Address. Upper 32 bits of address.
0x28	MM2S_LENGTH	MM2STransfer Length (Bytes)
0x30	S2MM_DMACR	S2MM DMA Control register
0x34	S2MM_DMASR	S2MM DMA Status register
0x48	S2MM_DA	S2MM Destination Address. Lower 32 bit address.
0x4C	S2MM_DA_MSB	S2MM Destination Address. Upper 32 bit address.
0x58	S2MM_LENGTH	S2MMBuffer Length (Bytes)

2.3 C kód

Egyetlen 128 byte-os átvitelt (olvasás és írás) megvalósító C kód (olvashatóan tördelve, syntax highlight-tal, SAJÁT kommentekkel):

```
#include <xparameters.h>
#include <stdint.h>
#include <xil_cache.h>
#include <xaxidma_hw.h>

#define MEM32(addr) (*(volatile uint32_t *) (addr))

#define DMA_BUFFER_SIZE (1024 * 1024)

uint8_t dma_tx_buf[DMA_BUFFER_SIZE] __attribute__((aligned(128)));
uint8_t dma_rx_buf[DMA_BUFFER_SIZE] __attribute__((aligned(128)));

// AXI GPIO leirasbol
#define DMA_BASE_ADDR 0xa0010000

void dma_init();
void dma_snd(void* src, uint32_t length);
void dma_rcv(void* dst, uint32_t length);
void dma_snd_wait();
void dma_rcv_wait();
void mem_cmp(void *buf1, void *buf2, uint32_t length);

int main() {

    dma_init(XPAR_AXI_DMA_0_BASEADDR);

    for(int i = 0; i < DMA_BUFFER_SIZE; ++i){
        dma_tx_buf[i] = (unsigned char) (i & 0xff);
        dma_rx_buf[i] = 0x00;
    }

    mem_cmp(dma_tx_buf, dma_rx_buf, DMA_BUFFER_SIZE);

    dma_snd(dma_tx_buf, 128);
    dma_rcv(dma_rx_buf, 128);
    dma_snd_wait();
    dma_rcv_wait();

    mem_cmp(dma_tx_buf, dma_rx_buf, 128);

    for(int i=0; i<1024/128;++i){
        dma_snd(dma_tx_buf + i * 128, 128);
        dma_rcv(dma_rx_buf + i * 128, 128);
    }
}
```

```

    dma_snd_wait();
    dma_rcv_wait();

    mem_cmp(dma_tx_buf, dma_rx_buf, 1024);

    dma_snd(dma_tx_buf, 1024);
    dma_rcv(dma_rx_buf, 1024);
    dma_snd_wait();
    dma_rcv_wait();

    mem_cmp(dma_tx_buf, dma_rx_buf, 1024);

}

void dma_init() {
    MEM32(DMA_BASE_ADDR + XAXIDMA_TX_OFFSET + XAXIDMA_CR_OFFSET) =
XAXIDMA_CR_RUNSTOP_MASK;
    MEM32(DMA_BASE_ADDR + XAXIDMA_RX_OFFSET + XAXIDMA_CR_OFFSET) =
XAXIDMA_CR_RUNSTOP_MASK;
}
void dma_snd(void* src, uint32_t length) {
    Xil_DCacheFlush();
    MEM32(DMA_BASE_ADDR + XAXIDMA_TX_OFFSET + XAXIDMA_SRCADDR_OFFSET)
= (uint32_t)src;
    MEM32(DMA_BASE_ADDR + XAXIDMA_TX_OFFSET + XAXIDMA_SRCADDR_MSB_OFFSET) = (uint32_t)((uintptr_t)src >> 32);
    MEM32(DMA_BASE_ADDR + XAXIDMA_TX_OFFSET + XAXIDMA_BUFFLEN_OFFSET)
= length;
}
void dma_rcv(void* dst, uint32_t length) {
    Xil_DCacheInvalidateRange((INTPTR)dst, (INTPTR)length);
    MEM32(DMA_BASE_ADDR + XAXIDMA_RX_OFFSET + XAXIDMA_SRCADDR_OFFSET)
= (uint32_t)dst;
    MEM32(DMA_BASE_ADDR + XAXIDMA_RX_OFFSET + XAXIDMA_SRCADDR_MSB_OFFSET) = (uint32_t)((uintptr_t)dst >> 32);
    MEM32(DMA_BASE_ADDR + XAXIDMA_RX_OFFSET + XAXIDMA_BUFFLEN_OFFSET)
= length;
}
void dma_snd_wait() {
    for(;;) {
        if((DMA_BASE_ADDR + XAXIDMA_TX_OFFSET + XAXIDMA_SR_OFFSET) &
XAXIDMA_IDLE_MASK) {
            break;
        }
    }
}
void dma_rcv_wait() {
    for(;;) {

```



```

        if((DMA_BASE_ADDR + XAXIDMA_RX_OFFSET + XAXIDMA_SR_OFFSET) &
XAXIDMA_IDLE_MASK) {
            break;
        }
    }
}

void mem_cmp(void *buf1, void *buf2, uint32_t length){
    uint8_t *ptr1 = (uint8_t *)buf1;
    uint8_t *ptr2 = (uint8_t *)buf2;
    uint32_t i;

    for (i = 0; i < length; i++)
    {
        if (ptr1[i] != ptr2[i])
        {
            xil_printf(
                "mem_cmp(): Compare error!
buf1[%d]=0x%02x, buf2[%d]=0x%02x\r\n",
                i,
                ptr1[i],
                i,
                ptr2[i]
            );
            return;
        }
    }

    xil_printf("mem_cmp(): Content of the buffers is the
same.\r\n");
}

```

Kérdések:

- Hol helyezkedik el a DMA két buffere (transmit és receive)?
 - Transmit (küldési) oldal: A küldési buffer a rendszer DDR memóriájában helyezkedik el; itt hozza létre és tárolja a processzor a továbbítandó adatsorozatot.
 - Receive (fogadási) oldal: A fogadási folyamat során az adatok forrása jelenleg az AXI Stream FIFO periféria, ahonnan a DMA vezérlő az adatokat a szintén a DDR memóriában kijelölt célterületre (bufferbe) mozgatja át.
- Az S2MM vagy MM2S irány felprogramozása történik először?

- A DMA vezérlő inicializálása során elsőként az S2MM (Stream to Memory Map), azaz a fogadási irány felprogramozása történik meg.
- A tervezési alapelv szerint az adatútban (datapath) elhelyezkedő blokkokat a nyelőtől (destination) a forrás (source) felé haladó sorrendben preferált konfigurálni.
- Általában miért preferált megoldás, hogy az adatútban szereplő blokkok felprogramozása a céltól a forrás felé haladó sorrendben történik?
 - Amint az utolsóként konfigurált elem, a forrás (source) is megkapja a paramétereit, az adatátvitel azonnal, várakozás nélkül megindulhat.

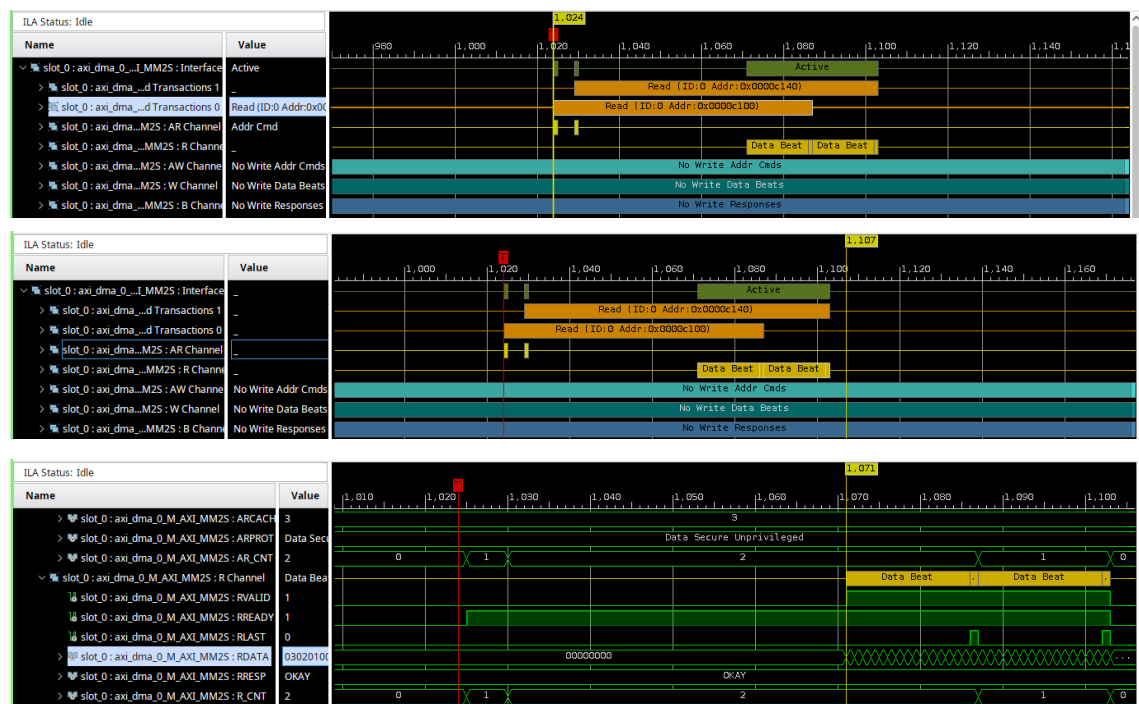
2.4 Átvitel vizsgálata ChipScope-pal

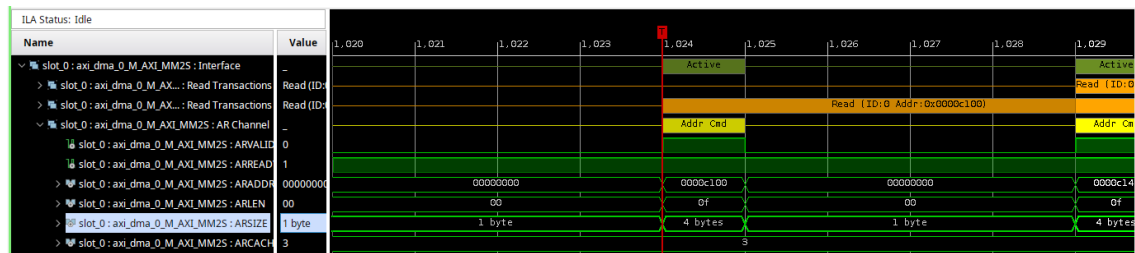
2.4.1 MM2S irány (Processzoros alrendszer → FIFO) vizsgálata egyetlen 128 byte-os átvitel esetén

Chipscope trigger beállítások

slot_0 : axi_dma_0_M_AXI_MM2S : ARVALID	==	▼	[B]	▼	1	▼	probe20[0]	1 c
slot_0 : axi_dma_0_M_AXI_MM2S : ARREADY	==	▼	[B]	▼	1	▼	probe20[1]	1 c

ChipScope hullámforma ÁBRA, a 2 lényeges interface összesen 3 channel-jével





Kérdések:

- Mennyi a teljes buszciklus ideje órajelben számolva?
 - A teljes buszciklus a mérés alapján az 1024. és az 1107. órajelciklus között zajlik le.
- Mennyi ebből a tényleges adatátvitel ideje? Hány százalék a busz hatékonysága?
 - Az interfészen a tényleges adatkiolvasás a processzorból az 1071. és az 1103. órajelciklusok között történik, míg a teljes vizsgált folyamat az 1024–1103. ciklusintervallumot öleli fel.
- Hány burst-ben történik a 128 byte átvitele? Miből látszik ez (2 megoldás)?
 - A ChipScope hullámformán leolvasható ARLEN paraméter értéke 0x0f. Az AXI protokoll szerint ez $15+1=16$ adatátviteli ütemet jelent burst-önként. Mivel az adatszélesség 4 byte (32 bit), egy burst során $16*4=64$ byte adat továbbítása történik meg, így a teljes 128 byte-os csomaghoz 2 burst szükséges.
 - Az ILA mérésen megfigyelhető, hogy az olvasási címcsatornán (Address Read) kétszer történik érvényes kézfogás az ARVALID és ARREADY jelek egyidejű magas szintjével, ami két különálló tranzakció indítását igazolja.
- Mekkora a címzés és a tényleges adatátvitel közötti késleltetés órajelben? Mi az oka ennek? Hogyan csökkenthetnénk?
 - A buszciklus kezdete (1024. órajel) és a tényleges adatmegjelenés (1071. órajel) között 47 órajelciklusnyi különbség mérhető.
 - Ezt az időt az AXI protokoll címzésének adminisztrációja, az AXI SmartConnect belső logikája (különösen szélesség-illesztés miatt), valamint a DDR memóriavezérlő elérési ideje okozza.
 - A késleltetés mérsékelhető az adatszélességek összehangolásával, vagy a memóriavezérlő sávszélesség-optimalizálásával.

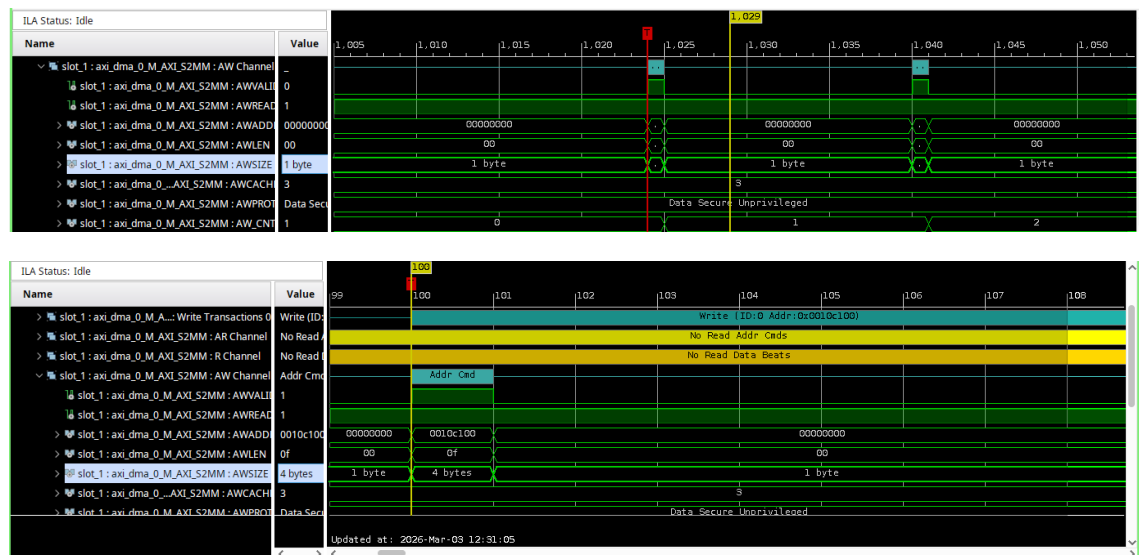
- Mennyi az AXI4 és az AXI4 Stream buszok közötti késleltetés? Mi lehet ennek az oka?
 - Az AXI4 (Memory Map) oldali adatmegjelenés (1071. órajel) és az AXI4-Stream oldali továbbítás kezdete (1075. órajel) között 4 órajelnyi időkülönbség tapasztalható.
 - Ezt a minimális késleltetést a DMA vezérlő belső pipeline regiszterei, valamint az adatok Stream formátumba való alakítása miatti feldolgozási idő okozza.

2.4.2 S2MM irány vizsgálata egyetlen 128 byte-os átvitel esetén

Chipscope trigger beállítások ÁBRA

slot_1 : axi_dma_0_M_AXI_S2MM : AWVALID	==	[B]	1	probe40[0]	1 c
slot_1 : axi_dma_0_M_AXI_S2MM : AWREADY	==	[B]	1	probe40[1]	1 c

ChipScope hullámforma ÁBRA, a 2 lényeges interfész összesen 3 channel-jével



Van-e lényeges különbség az MM2S irányhoz képest?

- Nincs

2.4.3 MM2S irány vizsgálata több 128 byte-os átvitel esetén

Chipscope trigger beállítások ÁBRA

Name	Operator	Radix	Value	Port	Co
slot_0 : axi_dma_0_M_AXI_MM2S : ARVALID	==	[B]	1	probe20[0]	1 c
slot_0 : axi_dma_0_M_AXI_MM2S : ARREADY	==	[B]	1	probe20[1]	1 c

ChipScope hullámforma ÁBRA, a 2 lényeges interface összesen 3 channel-jével



Kérdések:

- Hova célszerű állítani a trigger pozíciót, ha minél több egymás utáni átvitelt szeretnénk látni?
 - A trigger pozíciót érdemes a rögzítési puffer elejére, egy alacsony indexértékre (például a 100-as mintára vagy az alá) állítani.
- Kb. hány órajel telik el egy DMA befejezése és a következő DMA átvitel megkezdése között? Hogyan lehet ezt a legkisebbre csökkenteni?
 - A mérés alapján a két átvitel között körülbelül 200 órajelciklus (a 172. és 372. órajel közötti időkülönbség) telik el.
 - Ezt a holtidőt a Scatter-Gather (SG) mód alkalmazásával lehetne minimalizálni
 - A szoftveres overhead tovább csökkenthető a cache-kezelés optimalizálásával, például a cache-ürítési (Xil_DCacheFlush) parancs ciklusból való kiemelésével.

- Amennyiben az adatok egybefüggő területen helyezkednek el, elegendő a teljes tartományt a ciklus megkezdése előtt egyszer üríteni.
- A mérés alapján ezzel a módszerrel a tranzakciók közötti szünet 200-ról körülbelül 180 órajelre (170 és 350 közötti különbség) mérséklődött.
- A csökkentés megvalósítása utáni ChipScope hullámforma ÁBRA



2.4.4 S2MM irány vizsgálata több 128 byte-os átvitel esetén

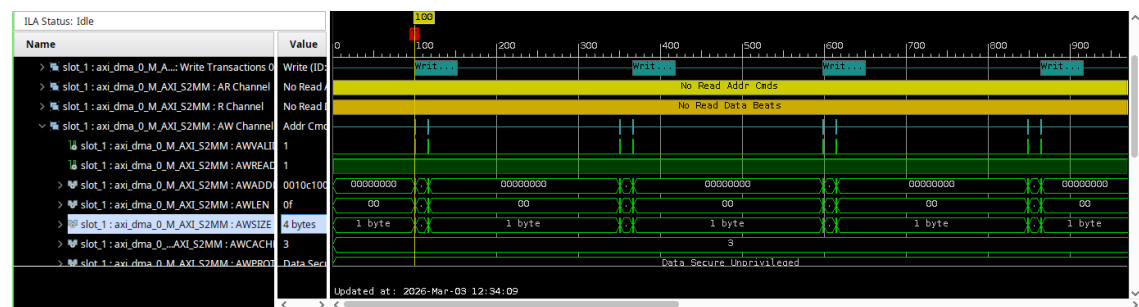
- Chipscope trigger beállítások ÁBRA

Name	Operator	Radix	Value	Port	Co
slot_0: axi_dma_0_M_AXI_S2MM : AWVALID ==	▼	[B]	▼	1	▼
slot_0: axi_dma_0_M_AXI_S2MM : AWREADY==	▼	[B]	▼	1	▼

- ILA Status: Idle

Name	Value
slot_1: axi_dma_0_M_AXI_S2MM: AWVALID	0
slot_1: axi_dma_0_M_AXI_S2MM: AWPROT	Data Secure
slot_1: axi_dma_0_M_AXI_S2MM: AW_CNT	0
slot_1: axi_dma_0_M_AXI_S2MM: W Channel	1
slot_1: axi_dma_0_M_AXI_S2MM: WVALID	0
slot_1: axi_dma_0_M_AXI_S2MM: WREADY	1
slot_1: axi_dma_0_M_AXI_S2MM: WLAST	0
slot_1: axi_dma_0_M_AXI_S2MM: WDATA	03020100
slot_1: axi_dma_0_M_AXI_S2MM: WSTRB	f
slot_1: axi_dma_0_M_AXI_S2MM: B Channel	-
slot_2: axi_dma_0_M_AXI_S2MM: Interface	-

Updated at: 2026-Mar-03 12:34:09



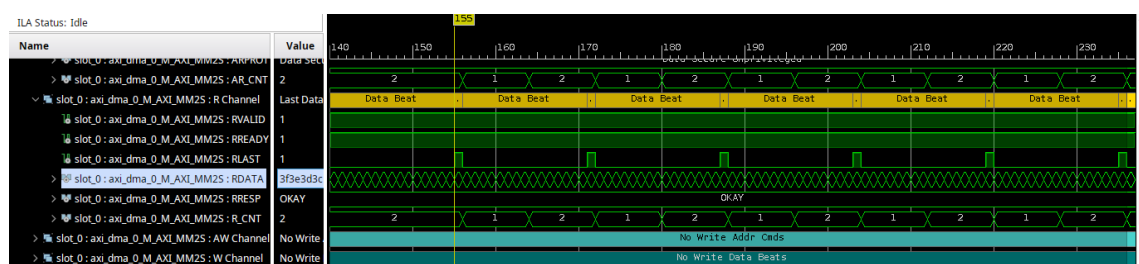
- ### 2.4.5 MM2S irány egyetlen 1024 byte-os blokk („packet”) átvitele során

Name	Operator	Radix	Value	Port	Co
slot_0 : axi_dma_0_M_AXI_MM2S : ARVALID	==	[B]	1	probe20[0]	1 d
slot_0 : axi_dma_0_M_AXI_MM2S : ARREADY	==	[B]	1	probe20[1]	1 d

ILA Status: Idle

Name	Value
slot_0:axi_dma_0_M_AXI_MM2S:Interface	
slot_0:axi_dma_0_M_AXI_MM2S:Read Transactions	OVERFLOW
slot_0:axi_dma_0_M_AXI_MM2S:AR Channel	-
slot_0:axi_dma_0_M_AXI_MM2S:ARVALID	0
slot_0:axi_dma_0_M_AXI_MM2S:ARREADY	1
slot_0:axi_dma_0_M_AXI_MM2S:ARADDR	00000000
slot_0:axi_dma_0_M_AXI_MM2S:ARLEN	00
slot_0:axi_dma_0_M_AXI_MM2S:ARSIZE	1 byte
slot_0:axi_dma_0_M_AXI_MM2S:ARCACh	3
slot_0:axi_dma_0_M_AXI_MM2S:ARPROT	Data Sec
slot_0:axi_dma_0_M_AXI_MM2S:AR_Cnt	

The waveform on the right shows a sequence of transactions over time. A red bar labeled 'OVERFLOW' indicates a data overflow event. The transactions are shown as green bars with various values (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683



- Van-e lényeges különbség az első (egyetlen 128 byte-os átvitelt vizsgáló) esethez képest?
 - Az 1024 byte-os blokk átvitele során az adatok továbbra is azonos méretű burstökben továbbítódnak, mint a 128 byte-os esetben, azonban a nagyobb adatmennyiség miatt a tranzakciók közötti holtidő aránya csökken, ami jelentősen mérsékli az összkésleltetést.

3 Audio interfész (I2S) megvalósítása

3.1 Left justified I2S átvitel hullámformája:

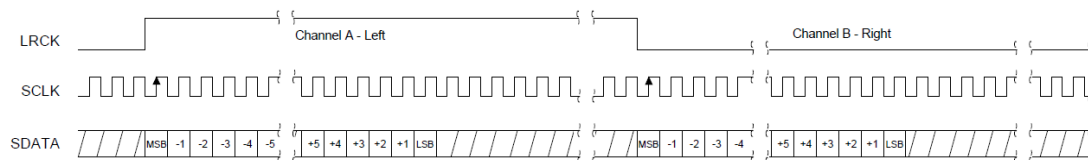


Figure 5. Format 0, Left-Justified up to 24-Bit Data

Kérdések:

- Az interfész realizálása szempontjából melyik adatátviteli mód (I2S, left justified, right justified) lenne a leghatékonyabb és miért?
 - A megvalósítás szempontjából a Left Justified mód a legelőnyösebb. Ebben a módban a csatornaváltást (LRCLK váltást) követő legelső órajelével megkezdhető az adatok bepörgetése a shift regiszterbe, míg a standard I2S szabvány esetén egy órajelciklusnyi várakozás szükséges. További előnye a Right Justified móddal szemben, hogy az adatokat MSB-aligned (legnagyobb helyiértékű bit szerint igazított) formában kapjuk meg, így a feldolgozás azonnal megkezdhető az alacsonyabb helyiértékű bitekre való várakozás nélkül.
- 96 kHz-es mintavételi frekvencia esetén mekkora a CODEC irányából érkező BCLK és LRCLK jelek frekvenciája? Detektálható-e ezen jelek éle megbízhatóan a 100 MHz-es rendszer órajellel?
 - LRCLK: Frekvenciája megegyezik a mintavételi frekvenciával, tehát 96 kHz.
 - BCLK: 32 bites adatszó-hosszúság és két csatorna (sztereó) esetén a frekvencia: $96 \text{ kHz} * 2 * 32 = 6,144 \text{ MHz}$.
 - Detektálhatóság: A 100 MHz-es rendszerórajel mellett mindkét jel éle megbízhatóan detektálható, mivel a mintavételi tétel feltételei teljesülnek. A rendszerórajel frekvenciája elegendő időt biztosít a

többfokozatú szinkronizáció végrehajtására is, ami kritikus a metastabilitás elkerülése érdekében.

3.2 A megvalósított modul HDL kódja

Az I2S interfészt megvalósító modul HDL kódja (olvashatóan törölve, syntax highlight-tal, kommentekkel):

```
module audio_interface(
    input clk,
    input mclk_in,
    input rst_n,
    input en,
    output codec_mclk,
    input codec_lrclk,
    input codec_bclk,
    input codec_sdout,
    output codec_sdin,
    output [23:0] adc_data,
    output adc_valid_l,
    output adc_valid_r
);

// Positive edge reset
wire rst;
assign rst = ~rst_n;

assign codec_sdin = 0;

reg [2:0] lrclk_shr;
reg [2:0] bclk_shr;
reg [1:0] sdout_shr;

always @ (posedge clk) begin
    if(rst) begin
        lrclk_shr <= 0;
        bclk_shr <= 0;
        sdout_shr <= 0;
    end
    else begin
        lrclk_shr <= {lrclk_shr[1:0], codec_lrclk};
        bclk_shr <= {bclk_shr[1:0], codec_bclk};
        sdout_shr <= {sdout_shr[0], codec_sdout};
    end
end

wire lrclk_rise = (lrclk_shr[2:1] == 2'b01);
wire lrclk_fall = (lrclk_shr[2:1] == 2'b10);
wire bclk_rise = (bclk_shr[2:1] == 2'b01);
wire bclk_fall = (bclk_shr[2:1] == 2'b10);

reg [31:0] adc_data_shr;
always @ (posedge clk) begin
    if(bclk_rise) begin
        adc_data_shr <= {adc_data_shr[30:0], sdout_shr[1]};
    end
end
```

```

        end
    end

    assign adc_valid_l = lrclk_fall & en;
    assign adc_valid_r = lrclk_rise & en;

    assign adc_data = adc_data_shr[31:8];

    ODDRE1 #(
        .IS_C_INVERTED(1'b0),           // Optional inversion for C
        .IS_D1_INVERTED(1'b0),          // Unsupported, do not use
        .IS_D2_INVERTED(1'b0),          // Unsupported, do not use
        .SRVAL(1'b0)                     // Initializes the ODDRE1 Flip-Flops
    to the specified value (1'b0, 1'b1)
    )
    ODDRE1_inst (
        .Q(codec_mclk), // 1-bit output: Data output to IOB
        .C(mclk_in),    // 1-bit input: High-speed clock input
        .D1(1'b1),      // 1-bit input: Parallel data input 1
        .D2(1'b0),      // 1-bit input: Parallel data input 2
        .SR(rst)        // 1-bit input: Active-High Async Reset
    );

endmodule

```

3.3 A tesztkörnyezet HDL kódja

A modul tesztelésére szolgáló egyszerű testbench kódja (olvashatóan tördelve, syntax highlight-tal, kommentekkel):

```

`timescale 1ns / 1ps

module audio_interface_tb;

    reg clk;
    reg mclk_in;
    reg rst_n;
    reg en;
    reg codec_lrclk;
    reg codec_bclk;
    reg codec_sdout;

    wire codec_mclk;
    wire codec_sdin;
    wire [23:0] adc_data;
    wire adc_valid_l;
    wire adc_valid_r;

    audio_interface uut (
        .clk(clk),
        .mclk_in(mclk_in),
        .rst_n(rst_n),
        .en(en),

```

```

        .codec_mclk(codec_mclk),
        .codec_lrclk(codec_lrclk),
        .codec_bclk(codec_bclk),
        .codec_sdout(codec_sdout),
        .codec_sdin(codec_sdin),
        .adc_data(adc_data),
        .adc_valid_l(adc_valid_l),
        .adc_valid_r(adc_valid_r)
    );

    // 100 MHz -> 10ns
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    rst_n = 0;
    en = 0;
    mclk_in = 0;
    codec_sdout = 0;

    #100;
    rst_n = 1;
    en = 1;
    #50000;
    $finish;
end

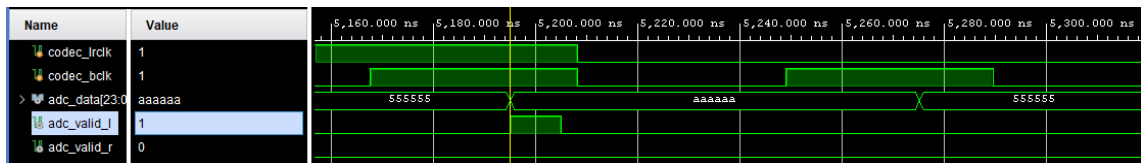
initial begin
    // 192kHz -> 5208.33ns
    codec_lrclk = 0;
    forever #2604.16 codec_lrclk = ~codec_lrclk;
end

initial begin
    // 192kHz * 32 * 2 = 12.288MHz -> 81.38ns
    codec_bclk = 0;
    forever #40.69 codec_bclk = ~codec_bclk;
end

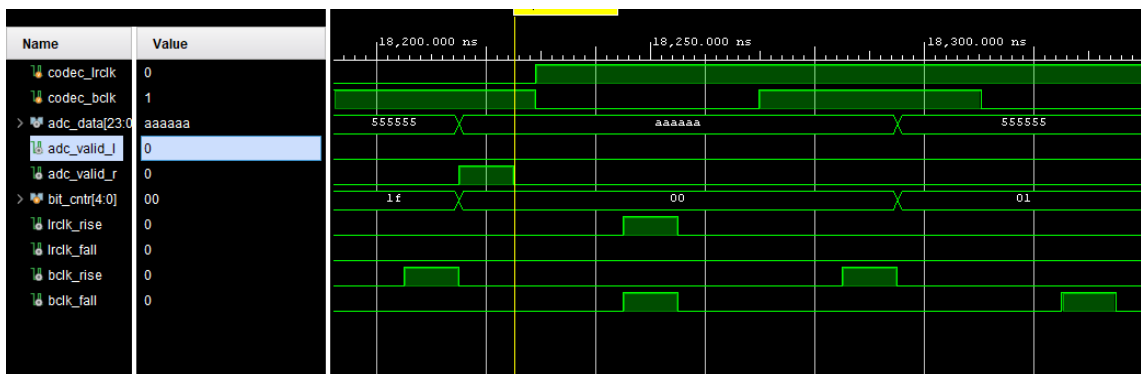
initial begin
    codec_sdout = 0;
    #150;
    // Generating 1s and 0s on the data line
    forever begin
        @(negedge codec_bclk);
        codec_sdout = ~codec_sdout;
    end
end
endmodule

```

3.4 Szimulációs hullámforma, magyarázattal:



- A szimulációs eredmények alapján megállapítható, hogy az `adc_valid_x` jel időzítése megfelelő: az állapotváltás még a csatornaváltást megelőzően bekövetkezik, amikor a shift regiszter tartalma már mind a 32 bitnyi aktuális adatot tartalmazza.



- A bemeneti jelek éleinek detektálása a háromfokozatú szinkronizáló lánc alkalmazása miatt 2-3 rendszerórajel-ciklusnyi késleltetést szenved el a fizikai jelekhez képest.
- Mivel az adatvonal szinkronizálása a vezérlőjelekkel (`LRCLK`, `BCLK`) párhuzamosan történik, ez a fáziskésés nem okoz működési hibát, a belső feldolgozás számára az adatok és a hozzájuk tartozó validáló jelek időzítése konzisztens marad.

4 Audió CODEC interfész

4.1 Konfigurációs interfész (I2C) megvalósítása

4.1.1 CODEC MCLK órajel generálása

Kérdések:

- A CODEC adatlapja alapján a 96 kHz-es mintavételi frekvencia eléréséhez mekkora MCLK órajel szükséges?

LRCK (kHz)	MCLK (MHz)								
	64x	96x	128x	192x	256x	384x	512x	768x	1024x
32	-	-	-	-	8.1920	12.2880	16.3840	24.5760	32.7680
44.1	-	-	-	-	11.2896	16.9344	22.5792	33.8680	45.1584
48	-	-	-	-	12.2880	18.4320	24.5760	36.8640	49.1520
64	-	-	8.1920	12.2880	16.3840	24.5760	32.7680	-	-
88.2	-	-	11.2896	16.9344	22.5792	33.8680	45.1584	-	-
96	-	-	12.2880	18.4320	24.5760	36.8640	49.1520	-	-
128	8.1920	12.2880	16.3840	24.5760	32.7680	-	-	-	-
176.4	11.2896	16.9344	22.5792	33.8680	45.1584	-	-	-	-
192	12.2880	18.4320	24.5760	36.8640	49.1520	-	-	-	-
Mode	QSM					DSM		SSM	

- Ebben az esetben az MCLK a mintavételi frekvenciának hányszorosa (64x, 128x, 256x, 512x vagy 1024x)?
 - Ebben az üzemmódban az MCLK a mintavételi frekvencia 256-szorosa ($256 * fs$). Az adatlap szerint Double-Speed módban a támogatott szorzók a 128, 192, 256, 384 és 512.
- Melyik módon valósította meg az MCLK órajel előállítását? Magyarázza el a választott megoldás működését.
 - Az órajelet a Clocking Wizard IP segítségével állítottam elő. A választott szorzó a 256x, mivel ez biztosította a legpontosabb illesztést és a legkisebb fáziszajt a szintézis során.

4.1.2 CODEC I2C interfész

ÁBRA: CODEC I2C írás időzítési diagram (CODEC adatlap 32. oldal)

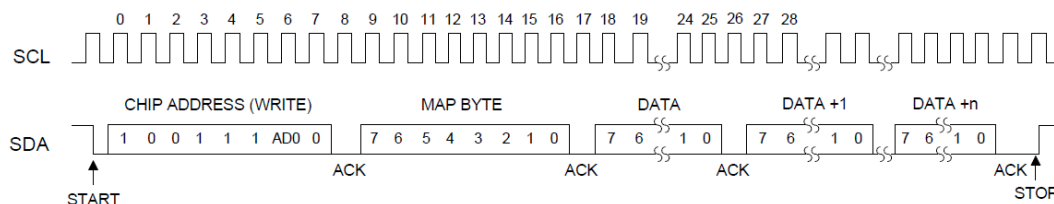


Figure 15. Control Port Timing, I²C Write

Kérdések:

- Mi azonosítja az I2C buszciklus kezdetét és végét?
 - A tranzakció kezdetét a START feltétel jelzi, ami az SDA vonal magasról alacsonyra váltása magas SCL szint mellett.
 - A ciklus végét a STOP feltétel azonosítja, amikor az SDA vonal alacsonyról magasra vált magas SCL órajel mellett.
- Mi az egyes byte-ok szerepe az időzítési diagramon?
 - Az első byte a 7 bites eszköz címet és az adatirányt jelző R/W bitet tartalmazza.
 - A második byte a CODEC belső regisztercímét határozza meg.
 - A harmadik byte a regiszterbe írandó vagy onnan kiolvasott adatot hordozza.
- Mik az eszköz azonosító lehetséges értékei (chip address)?
 - A CODEC címe binárisan 100101[AD0]. Ha az AD0 láb alacsony szinten van, az eszköz címe 0x94 (írás) vagy 0x95 (olvasás).
- Hány bites a CODEC regisztereinek címe?
 - A CODEC belső regiszterei 8 bites címmel érhetők el.
- Milyen típusú interfész az I2C? Kell-e az FPGA-ban felhúzó ellenállás az SCL és SDA vonalakra? Miért?
 - Az I2C egy Open-Drain típusú interfész, amelynek a vonalakat a meghajtók csak alacsony szintre tudják húzni.
 - Emiatt mind az SDA, mind az SCL vonalon szükséges felhúzó ellenállás (akár külső, akár belső pull-up) a logikai magas szint biztosításához.

ÁBRA: CODEC I2C olvasási diagram (CODEC adatlap 32. oldal)

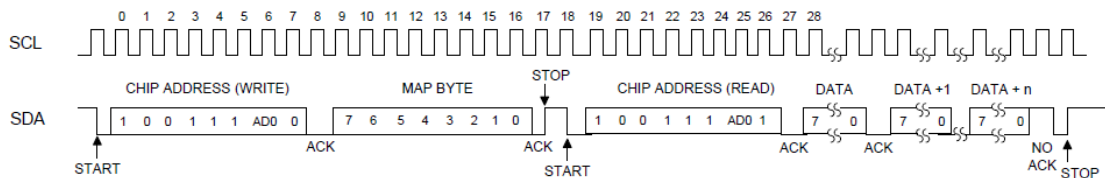


Figure 16. Control Port Timing, I²C Read

Kérdések:

- I2C busz esetén hogyan történik a tetszőleges címről történő olvasás? Azaz mi ez egyes byte-ok szerepe?
 - A tetszőleges címről való olvasás egy írási fázissal kezdődik a belső regisztermutató beállításához, amelyet egy ismételt START feltétel után az adatok tényleges beolvasása követ. Az első byte-ok az eszköz azonosítását és a belső regiszter kijelölését végzik, míg az irányváltás utáni byte-ok a kért adat átviteléért és a tranzakció lezárásáért felelősek.

4.2 Audió CODEC konfiguráció és ellenőrzés

4.2.1 Xilinx I2C periféria programozása

Kérdések:

- Vázlatosan ismertesse, hogy mi a különbség a Xilinx I2C periféria „Standard Controller Logic Flow” és a használt „Dynamic Controller Logic Flow” között?
 - Míg a Standard mód rögzített hardveres szekvenciákat követ, a Dynamic mód lehetővé teszi, hogy a szoftver a Transmit FIFO-ba írt specifikus parancsokkal közvetlenül vezérelje az I2C busz állapotgépét. Ez a rugalmasság elengedhetetlen az összetett tranzakciók (például ismételt START) megvalósításához anélkül, hogy a vezérlő elveszítené a busz feletti uralmat.
- Mely regiszter címeket kell használni „Dynamic Controller Logic Flow” alkalmazása során (lásd útmutató, illetve Xilin IIC dokumentáció 36. oldal)?
 - A dinamikus vezérlés alapvető regiszterei a Control Register (0x100), a Status Register (0x104), a TX_FIFO (0x108) és az RX_FIFO (0x10C).
- Hány bites a transmit, illetve receive FIFO? Ebből hány bit a vezérlés (mik ezek a bitek), és hány bit az I2C SDA vonalra kerülő érték?

- A Transmit és Receive FIFO-k egyaránt 10 bit szélességűek. Az alsó 8 bit (7:0) hordozza az SDA vonalra kerülő tényleges adatot, míg a felső 2 bit (9:8) a START és STOP feltételek generálásáért felelős vezérlőbitként funkcionál.

4.2.2 CODEC regiszterek beállítása

A mérési útmutatóban nem megadott regiszter értékek meghatározása. A táblázatokat bővítse szükség szerint.

Kérdések:

- A 4.1 pontban meghatározott órajelek használatához milyen értékekkel kell felprogramozni a CODEC megfelelő regisztereit (a mérési útmutatóban nem megadott regiszterek)?

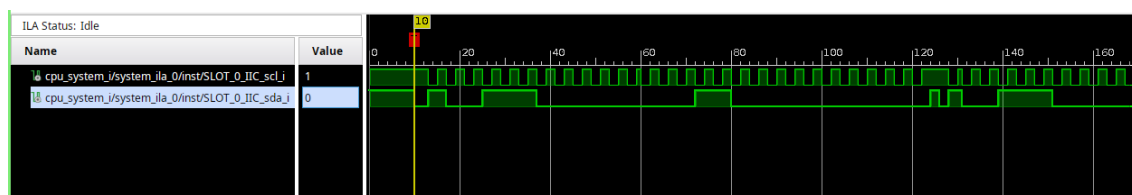
Regiszter cím	Érték	A beállított érték hatása
0x04	x41	Double speed mode
0x05	x20	Megfelelő MCLK osztás

- Hogyan kell felprogramozni a CODEC megfelelő regisztereit ahhoz, hogy az I2S interfész „left justified” módban működjön?

Regiszter cím	Érték	A beállított érték hatása
0x03	0x00	Ez állítja be a left justified módot

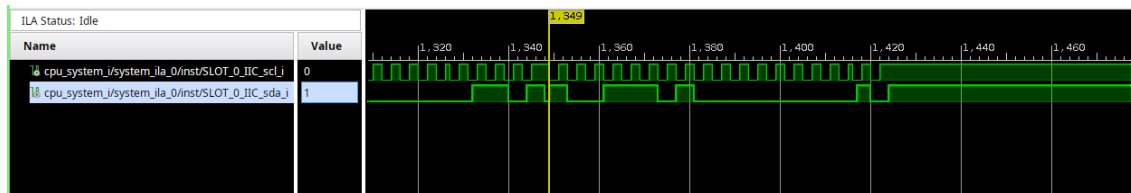
4.2.3 I2C átvitel ellenőrzése ChipScope-ban

ÁBRA: Egy I2C írás hullámformája. Megfelel a hullámforma az elvártnak?



- A rögzített hullámforma maradéktalanul megfelel az elvárt működésnek: a tranzakció a START feltétellel kezdődik, amelyet a 7 bites eszköz cím (Slave Address) és az írási iránybit (Write) követ.
- Megfigyelhető a periféria által küldött nyugtázó jel (ACK), majd a célregiszter címének átvitele, amelyet szintén nyugtázás, végül a tényleges adatbájt továbbítása és a STOP feltétel követ.

ÁBRA: Egy I2C olvasás hullámformája. Megfelel a hullámforma az elvártnak?



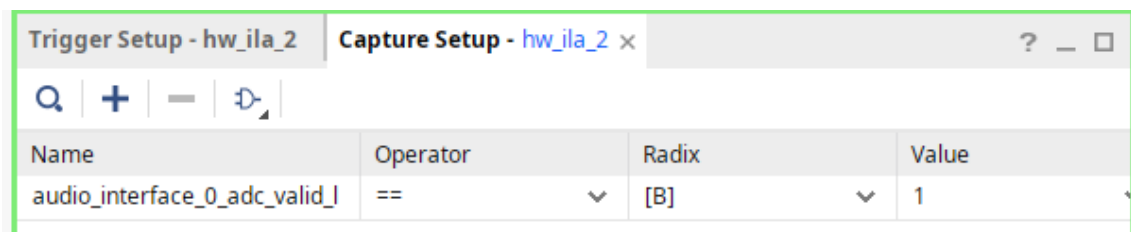
- A hullámforma megfelel az elméleti időzítési diagramnak: mivel a korábban 0x00 értékkel felprogramozott 0x03-as regisztert szólítjuk meg, a buszon a várt válasz érkezik.
- A tranzakció során látható a START feltétel és az eszköz cím kiadása, majd az adatfázisban a CODEC által visszaküldött 0x00 érték.
- Az olvasási ciklus lezárásaként a mester (FPGA) által kiadott negatív nyugtázás (NACK) és a buszt felszabadító STOP feltétel is azonosítható.

4.2.4 I2S interfész ellenőrzése

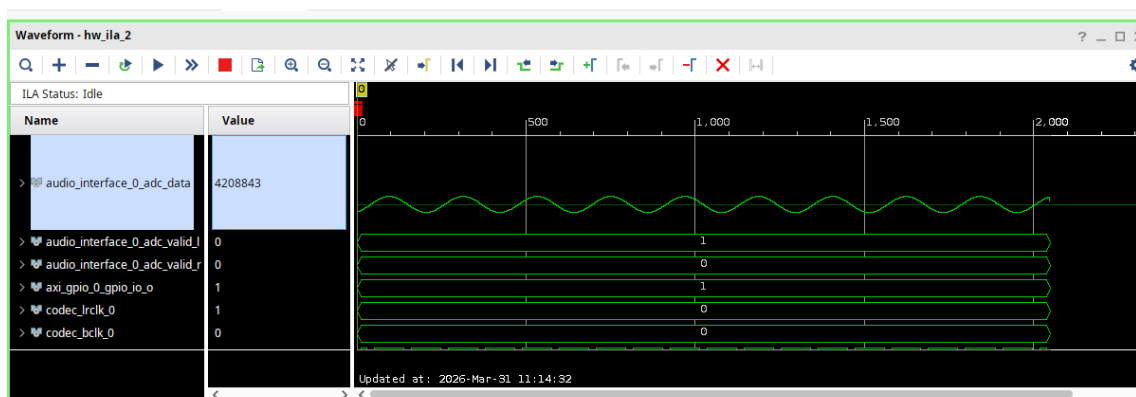
Kösse össze a PC line-out kimenetét a kodek modul vonali bemenetével, majd generáljon PC-n szinusz jelet (pl. Audacity). Megfelelő ChipScope beállítások használatával ellenőrizze, hogy az I2S interfész kimenetének hullámformája megfelel-e a generált jelnek.

ÁBRA: ChipScope trigger beállítás

ÁBRA: ChipScope Storage beállítás



ÁBRA: ChipScope hullámforma



5 I2S → AXI Stream interfész megvalósítása Vitis HLS-ben

5.1 Vivado HLS

Vitis HLS kód (+syntax highlight, komment)

```
#include "ap_int.h"
#include "ap_fixed.h"
#include "ap_axi_sdata.h"
#include "hls_stream.h"

typedef ap_fixed<24, 1, AP_TRN, AP_WRAP> din_t;
typedef ap_fixed<32, 1, AP_TRN, AP_WRAP> dout_t;
typedef hls::axis<dout_t, 0, 0, 0> axis_type;
typedef hls::stream<axis_type> stream_type;

void fir_hw(ap_uint<16> tlast_dnum, din_t* input_l, din_t* input_r,
stream_type& res){
#pragma HLS INTERFACE mode=s_axilite port=tlast_dnum
#pragma HLS INTERFACE mode=ap_hs port=input_l
#pragma HLS INTERFACE mode=ap_hs port=input_r
#pragma HLS INTERFACE mode=axis port=res
#pragma HLS INTERFACE mode=ap_ctrl_none port=return

    static ap_uint<16> cnt = 0;
    axis_type out_data;

    // Left channel
    cnt++;
    out_data.data = (dout_t)*input_l;
    out_data.last = (cnt == tlast_dnum);
    out_data.keep = -1;
    res.write(out_data);

    if(cnt == tlast_dnum){
        cnt = 0;
    }

    // Right channel
    cnt++;
    out_data.data = (dout_t)*input_r;
    out_data.last = (cnt == tlast_dnum);
    out_data.keep = -1;
    res.write(out_data);

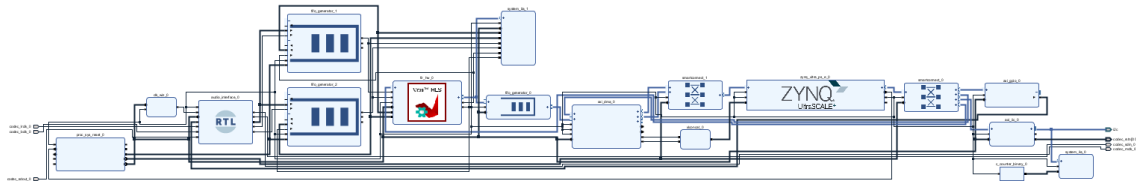
    if(cnt == tlast_dnum){
        cnt = 0;
    }
}
```

Vitis HLS fordítás eredményei (időzítés, erőforrás)

Performance & Resource Estimates										
MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM	
bypass_hw	20.000	3	no	function	0	0	96	275	0	

5.2 IP beillesztése a rendszerbe

ÁBRA: Block Design blokkvázlat



ÁBRA: Block Design címkiosztás

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0 (/axi_dma_0/Data_MM2S, /axi_dma_0/Data_S2MM)					
/axi_dma_0					
/axi_dma_0/Data_MM2S (32 address bits : 4G)					
/zynq_ultra_ps_e_0/SAXIGP0	S_AXI_HPC0_FPD	HPC0_DDR_LOW	0x0	2G	0x7FFF_FFFF
Excluded (3)					
/zynq_ultra_ps_e_0/SAXIGP0	S_AXI_HPC0_FPD	HPC0_DDR_HIGH			
/zynq_ultra_ps_e_0/SAXIGP0	S_AXI_HPC0_FPD	HPC0_QSPI	0xC000_0000	512M	0xDFFF_FFFF
/zynq_ultra_ps_e_0/SAXIGP0	S_AXI_HPC0_FPD	HPC0_LPS_OCM	0xFF00_0000	16M	0xFFFF_FFFF
/axi_dma_0/Data_S2MM (32 address bits : 4G)					
/zynq_ultra_ps_e_0/SAXIGP0	S_AXI_HPC0_FPD	HPC0_DDR_LOW	0x0	2G	0x7FFF_FFFF
Excluded (3)					
/zynq_ultra_ps_e_0/SAXIGP0	S_AXI_HPC0_FPD	HPC0_DDR_HIGH			
/zynq_ultra_ps_e_0/SAXIGP0	S_AXI_HPC0_FPD	HPC0_QSPI			
/zynq_ultra_ps_e_0/SAXIGP0	S_AXI_HPC0_FPD	HPC0_LPS_OCM			
Network 1 (/zynq_ultra_ps_e_0/Data)					
/zynq_ultra_ps_e_0					
/zynq_ultra_ps_e_0/Data (40 address bits : 0x00A0000000 [256M], 0x0400000000 [4G], 0x1000000000 [224G])					
/axi_dma_0/S_AXI_LITE	S_AXI_LITE	Reg	0xA001_0000	64K	0xA001_FFFF
/axi_gpio_0/S_AXI	S_AXI	Reg	0xA000_0000	64K	0xA000_FFFF
/axi_iic_0/S_AXI	S_AXI	Reg	0xA002_0000	64K	0xA002_FFFF
/fir_hw_0/s_axi_control	s_axi_control	Reg	0xA003_0000	64K	0xA003_FFFF

5.3 Tesztelés

A teszteléshez használt C kód forráskódja (+syntax highlight, komment)

```
#include <cmath>
#include <iostream>
#include <fstream>
#include <string>
#include <array>

#include "types.h"

constexpr unsigned int smpl_rd_num = 1;
constexpr unsigned int coeff_size = (1 << (smpl_rd_num - 1)) * 128;

int main(int argc, char* argv[]){
```

```

    std::string coeff_file_name = "./coefficients/filter_" +
std::to_string(coeff_size) + ".fcf";
    std::ifstream coeff_file(coeff_file_name);
    if (!coeff_file.is_open()) {
        std::cerr << "ERROR: Could not open " << coeff_file_name <<
std::endl;
        return -1;
    }
    std::array<coeff_t, 512> coeff_hw{0};

    std::string line;
    int count = 0;

    while(std::getline(coeff_file, line)){
        std::stringstream ss(line);
        double temp_val;

        if (ss >> temp_val) {
            if (count < 512) {
                coeff_hw[count] = (coeff_t)temp_val;
                count++;
            }
        }
    }

    coeff_file.close();
    std::cout << "Successfully loaded " << count << " coefficients."
<< std::endl;

    std::array<din_t, coeff_size> input_data{0};
    std::array<dout_t, coeff_size> res_l{0};
    input_data[0].range() = 0x7FFFFFFF;
    stream_type res_stream;

    for(int i = 0; i < input_data.size(); ++i) {
        bypass_hw(
            (ap_uint<16>)512,
            (ap_uint<3>)smp1_rd_num,
            (ap_uint<9>)(count - 1),
            coeff_hw.data(),
            &input_data[i],
            &input_data[i],
            res_stream
        );

        while (!res_stream.empty()) {
            axis_type out_val = res_stream.read();
            static bool is_left = true;
            if (is_left) {
                if(i < coeff_size) res_l[i] = out_val.data;
            }
            is_left = !is_left;
        }
    }

    std::cout << "\n--- Comparison: Coeff vs Impulse Response ---" <<
std::endl;

```

```

    int errors = 0;
    for(int i = 0; i < count; i++) {
        double diff = std::abs(coeff_hw[i].to_double() -
res_l[i].to_double());

        if (diff > 0.0000001) {
            std::cout << "Mismatch at [" << i << "]: Expected " <<
coeff_hw[i]
                        << " got " << res_l[i] << " (Diff: " << diff <<
")" << std::endl;
            errors++;
        }
    }

    if (errors == 0) {
        std::cout << "SUCCESS: Impulse response matches coefficients
exactly!" << std::endl;
    } else {
        std::cout << "FAILURE: Found " << errors << " mismatches." <<
std::endl;
    }

    return (errors == 0) ? 0 : 1;

    return 0;
}

```

5.3.1 A működés vizsgálata ChipScope-ban

ÁBRA: trigger beállítások

Trigger Setup - hw_ila_2

Capture Setup - hw_ila_2 x

?

—

□

Q

+

—

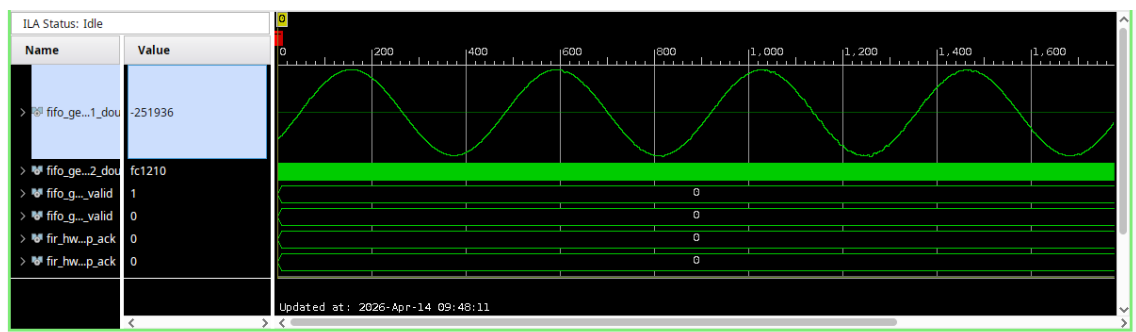
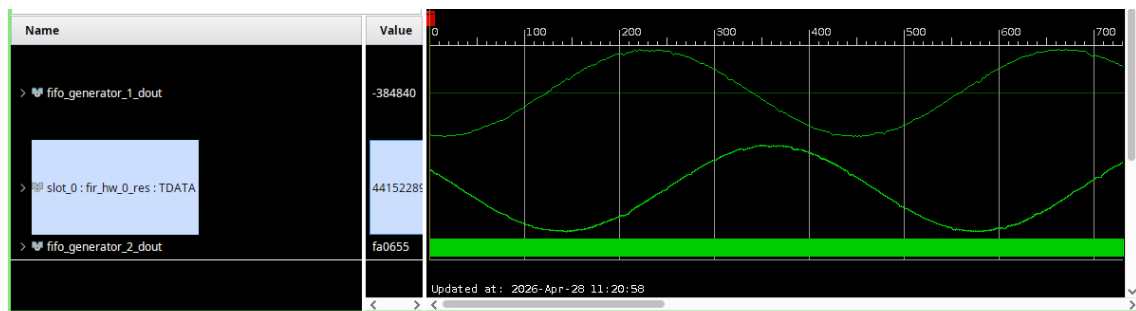
↺

Name	Operator	Radix	Value
slot_0 : fir_hw_0_res : TREADY	==	[B]	1
slot_0 : fir_hw_0_res : TVALID	==	[B]	1

<

>

ÁBRA: ChipScope hullámforma



6 FIR szűrő megvalósítása Vitis HLS-ben

- Figyelembe véve az audió mintavételi frekvenciát (96 kHz, sztereó) és az FPGA működési frekvenciáját (100 MHz) hány órajel áll rendelkezésre egyetlen kimeneti minta előállításához 1x, 2x és 4x decimálás esetén?
 - A 100 MHz-es rendszerórajel és a 96 kHz-es mintavételi frekvencia aránya alapján egyetlen kimeneti minta kiszámítására 1x decimálás esetén ~1041, 2x decimálásnál ~2083, 4x decimálásnál pedig ~4166 órajelciklus áll rendelkezésre.
- A szűrő fokszámát figyelembe véve hány MAC egység megvalósítását teszi ez szükségessé?
 - Logikailag összesen 2 MAC egység alkalmazása elegendő, mivel a rendelkezésre álló órajelek magas száma (~1041 ciklus/minta) lehetővé teszi a szűrőegyütthatók időosztásos feldolgozását.

6.1 A FIR szűrő C kódja prag mákkal, a prag mák magyarázatával

```
#include "types.h"

void fir_hw(ap_uint<16> tlast_dnum, ap_uint<3> smpl_rd_num, ap_uint<9>
tap_num_m1,
coeff_t coeff_hw[512], din_t *input_l, din_t *input_r, stream_type
&res){
#pragma HLS INTERFACE mode=s_axilite port=coeff_hw
#pragma HLS INTERFACE mode=s_axilite port=tap_num_m1
#pragma HLS INTERFACE mode=s_axilite port=smpl_rd_num
#pragma HLS INTERFACE mode=s_axilite port=tlast_dnum
#pragma HLS INTERFACE mode=ap_hs port=input_l
#pragma HLS INTERFACE mode=ap_hs port=input_r
#pragma HLS INTERFACE mode=axis port=res
#pragma HLS INTERFACE mode=ap_ctrl_none port=return

    static din_t buffer_left[512];
    static din_t buffer_right[512];
    static ap_uint<9> write_idx = 0;

    static ap_uint<16> cnt = 0;
    static ap_uint<3> dec_cnt = 0;
    axis_type out_data;

    buffer_left[write_idx]=(din_t)*input_l;
    buffer_right[write_idx]=(din_t)*input_r;
```



```

dec_cnt++;
if(dec_cnt >= smpl_rd_num){
    dec_cnt = 0;

    acc_t acc_l = 0;
    acc_t acc_r = 0;

    for(int i = 0; i <= tap_num_m1; ++i){
        #pragma HLS PIPELINE II=2
        #pragma HLS LOOP_TRIPCOUNT max=512 min=128
        ap_uint<9> read_idx = write_idx - i;
        din_t sample_l = buffer_left[read_idx];
        din_t sample_r = buffer_right[read_idx];
        acc_l += (acc_t)sample_l * coeff_hw[i];
        acc_r += (acc_t)sample_r * coeff_hw[i];
    }

    // Left channel
    cnt++;
    out_data.data = acc_l;
    out_data.last = (cnt == tlast_dnum);
    out_data.keep = -1;
    res.write(out_data);

    if(cnt == tlast_dnum){
        cnt = 0;
    }

    // Right channel
    cnt++;
    out_data.data = acc_r;
    out_data.last = (cnt == tlast_dnum);
    out_data.keep = -1;
    res.write(out_data);

    if(cnt == tlast_dnum){
        cnt = 0;
    }
}

++write_idx;

}

```

6.2 A C testbench forráskódja, az ellenőrzés eredménye

```

#include <cmath>
#include <iostream>
#include <fstream>
#include <string>
#include <array>

#include "types.h"

```

```

constexpr unsigned int smpl_rd_num = 1;
constexpr unsigned int coeff_size = (1 << (smpl_rd_num - 1)) * 128;

int main(int argc, char* argv[]){
    std::string coeff_file_name = "./coefficients/filter_" +
std::to_string(coeff_size) + ".fcf";
    std::ifstream coeff_file(coeff_file_name);
    if (!coeff_file.is_open()) {
        std::cerr << "ERROR: Could not open " << coeff_file_name <<
std::endl;
        return -1;
    }
    std::array<coeff_t, 512> coeff_hw{0};

    std::string line;
    int count = 0;

    while(std::getline(coeff_file, line)){
        std::stringstream ss(line);
        double temp_val;

        if (ss >> temp_val) {
            if (count < 512) {
                coeff_hw[count] = (coeff_t)temp_val;
                count++;
            }
        }
    }

    coeff_file.close();
    std::cout << "Successfully loaded " << count << " coefficients."
<< std::endl;

    std::array<din_t, coeff_size> input_data{0};
    std::array<dout_t, coeff_size> res_l{0};
    input_data[0].range() = 0x7FFFFFFF;
    stream_type res_stream;

    for(int i = 0; i < input_data.size(); ++i) {
        fir_hw(
            (ap_uint<16>)512,
            (ap_uint<3>)smpl_rd_num,
            (ap_uint<9>)(count - 1),
            coeff_hw.data(),
            &input_data[i],
            &input_data[i],
            res_stream
        );

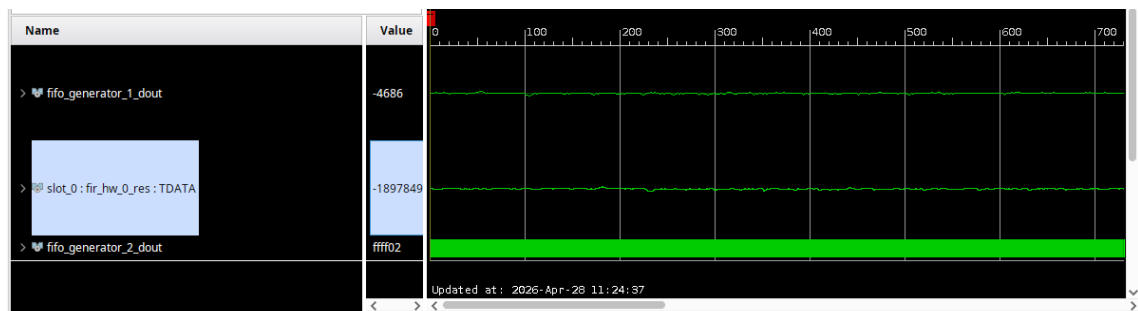
        while (!res_stream.empty()) {
            axis_type out_val = res_stream.read();
            static bool is_left = true;
            if (is_left) {
                if(i < coeff_size) res_l[i] = out_val.data;
            }
            is_left = !is_left;
        }
    }
}

```

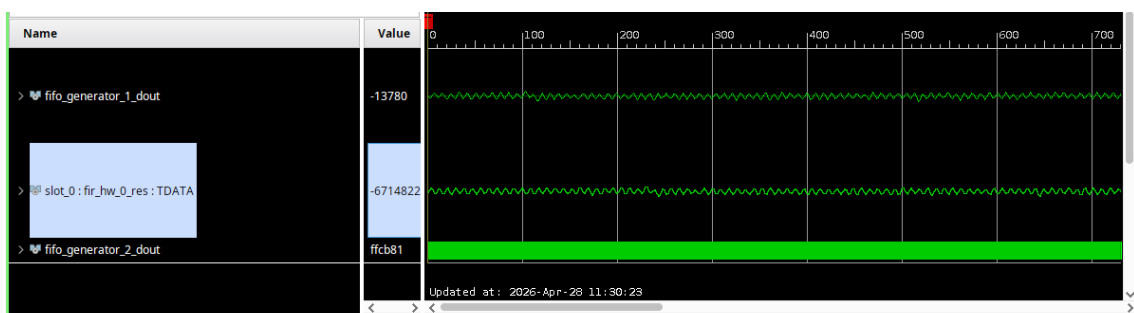

Name	Operator	Radix	Value
slot_0 : fir_hw_0_res : TREADY	==	[B]	1
slot_0 : fir_hw_0_res : TVALID	==	[B]	1

ÁBRA: ChipScope hullámforma decimálás nélkül

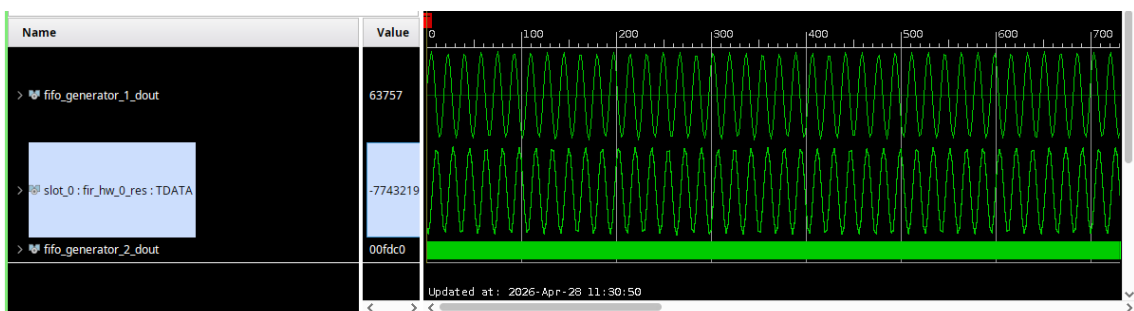
- 44kHz:



- 22kHz:

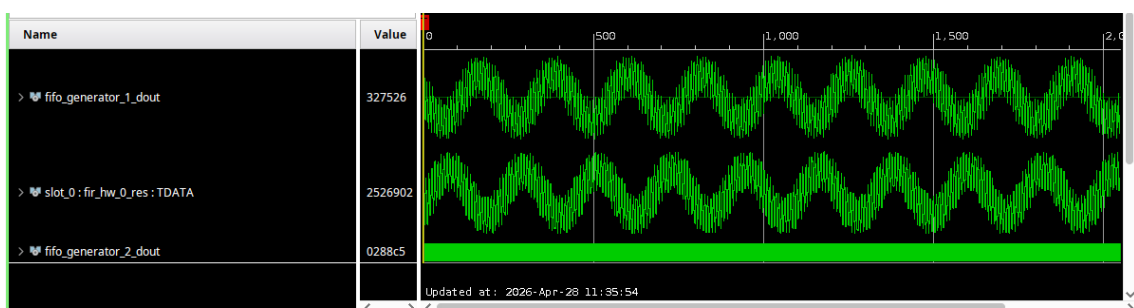


- 11kHz:

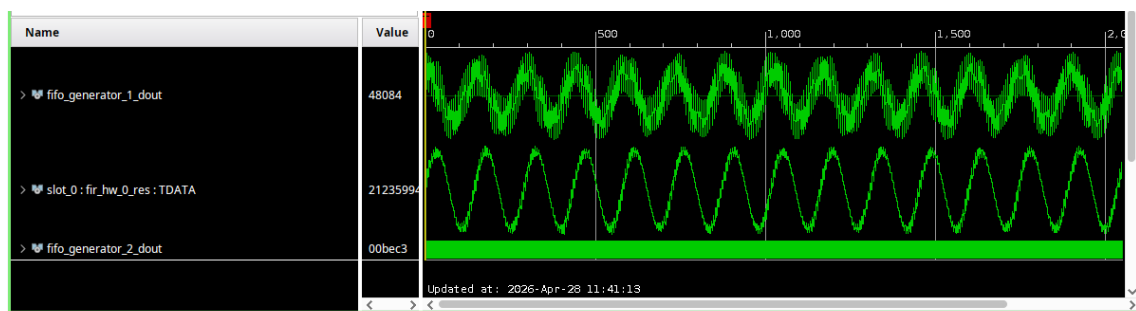


ÁBRA: ChipScope hullámforma decimálással, multiszinusz gerjesztéssel

- 440 Hz + 11 kHz + 22 kHz + 44 kHz:
 - 2x decimálás:



- 4xdecimálás:



7 A hálózati kommunikáció megvalósítása

7.1 A megvalósított dma_it_init() függvény kommentezett forráskódja

```
int dma_it_init(unsigned char *buff, unsigned long buff_size, unsigned
long transfer_size) {
    XAxiDma_Config *Config;
    int Status;

    g_buff = buff;
    g_buff_size = buff_size;
    g_transfer_size = transfer_size;
    dma_ptr_axi = g_buff;

    Config = XAxiDma_LookupConfig(XPAR_XAXIDMA_0_BASEADDR);
    if (!Config) return XST_FAILURE;

    Status = XAxiDma_CfgInitialize(&AxiDma, Config);
    if (Status != XST_SUCCESS) return Status;

    Status = XSetupInterruptSystem(&AxiDma, (void*)RxIntrHandler,
                                   Config->IntrId[1], Config->IntrParent,
                                   XINTERRUPT_DEFAULT_PRIORITY);
    if (Status != XST_SUCCESS) return Status;

    XAxiDma_IntrEnable(&AxiDma, XAXIDMA_IRQ_IOC_MASK,
XAXIDMA_DEVICE_TO_DMA);

    // Start the very first transfer
    return XAxiDma_SimpleTransfer(&AxiDma, (UINTPTR)dma_ptr_axi,
g_transfer_size, XAXIDMA_DEVICE_TO_DMA);
}
```

7.2 A megvalósított RxIntrHandler() függvény kommentezett forráskódja

```
void RxIntrHandler(void *Callback) {
    XAxiDma *AxiDmaInst = (XAxiDma *)Callback;

    // 1. Read and acknowledge interrupts
    u32 IrqStatus = XAxiDma_IntrGetIrq(AxiDmaInst,
XAXIDMA_DEVICE_TO_DMA);
    XAxiDma_IntrAckIrq(AxiDmaInst, IrqStatus,
XAXIDMA_DEVICE_TO_DMA);

    if (!(IrqStatus & XAXIDMA_IRQ_IOC_MASK)) return;

    // 2. Hand-off filled segment to the Ethernet task
```

```

        dma_ptr_eth = dma_ptr_axi;
        dma_data_available = 1;

        // 3. Move AXI pointer to the next block in the circular
buffer
        dma_ptr_axi += g_transfer_size;
        if (dma_ptr_axi >= (g_buff + g_buff_size)) {
            dma_ptr_axi = g_buff; // Wrap around to start
        }

        // 4. Trigger next transfer immediately to prevent data loss
        XAxiDma_SimpleTransfer(AxiDmaInst, (UINTPTR)dma_ptr_axi,
g_transfer_size, XAXIDMA_DEVICE_TO_DMA);
    }
}

```

7.3 A módosított transfer_data() függvény kommentezett forráskódja

```

int transfer_data() {
    if(audio_transfer_enabled && active_pcb && dma_data_available){

        //          static uint32_t sawtooth_counter = 0;
        //          uint32_t *samples = (uint32_t *)audio_send_buffer;
        //
        //          for (int i = 0; i < 256; i++) {
        //              samples[i] = sawtooth_counter;
        //              sawtooth_counter += 0x01000000;
        //          }

        Xil_DCacheInvalidateRange((UINTPTR)dma_ptr_eth, 1024);

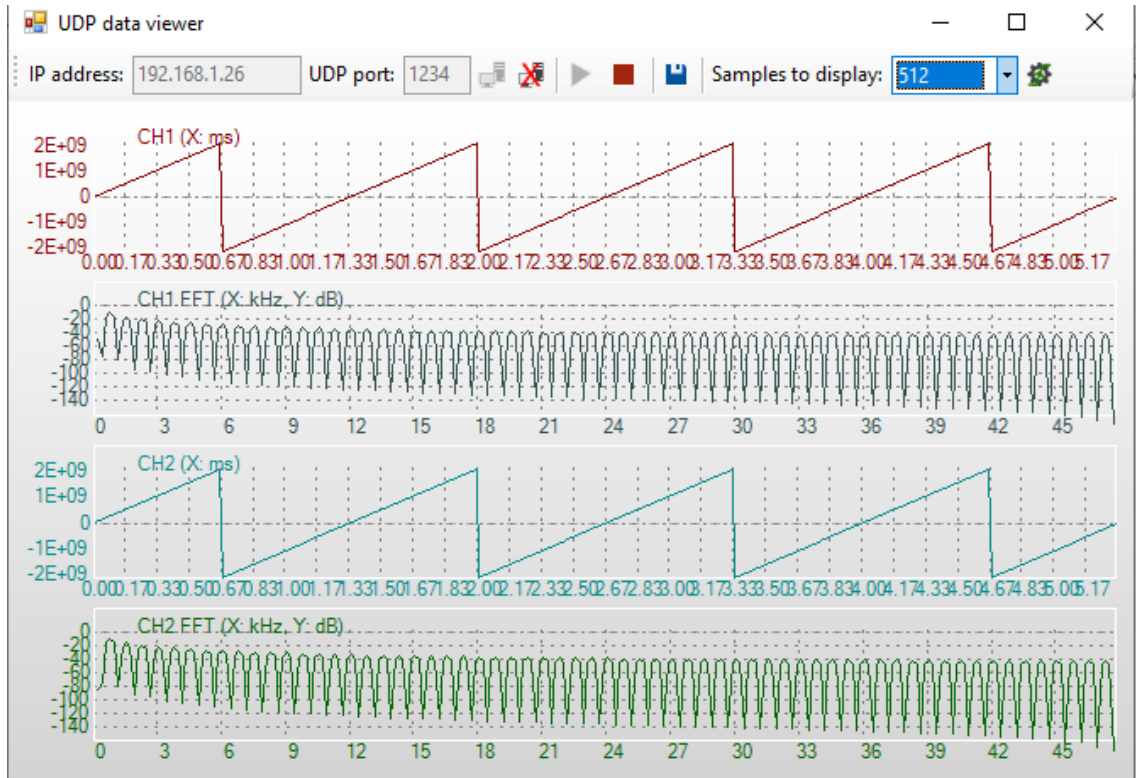
        struct pbuf *p = pbuf_alloc(PBUF_TRANSPORT, 1024,
PBUF_REF);
        if(p) {
            p->payload = dma_ptr_eth;
            err_t err = udp_sendto(active_pcb, p,
&remote_ip, remote_port);
            if (err != ERR_OK) {
                xil_printf("UDP Send Error: %d\n\r", err);
            }
            pbuf_free(p);
        } else {
            xil_printf("Could not allocate pbuf for
transmission\n\r");
        }

        dma_data_available = 0;
        new_data_available = 0;
    }
    return 0;
}

```

7.4 A helyes működést demonstráló képernyőkép a PC-s alkalmazásról

- Válasz „hardcoded” fűrészfél esetén:



- Válasz rendes működés mellett egy szinuszos gerjesztésre

